Merge K Sorted Lists

Desafio de algoritmos de entrevistas de big tech · LeetCode #23



Bernardo Fernandes Filho · Caio Vitor Soares · Vitor Mendonça Calicchio

O DESAFIO

O problema em detalhe

Entrada

Um array com K listas ligadas, cada uma já ordenada de forma crescente.

Saída

A cabeça de uma única lista ligada ordenada, com todos os nós.

Limites (LeetCode #23)

$0 \le K \le 10^4$ · até 500 nós por lista · até 10^4 nós no total

Cada nó da lista é assim:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

N = total de nós · K = número de listas

O DESAFIO

Montando o resultado

Passo	Cabeças das listas	Menor	Resultado até agora
1	1 · 1 · 2	**1 (lista 1)**	1
2	4 · 1 · 2	**1 (lista 2)**	1 → 1
3	4 · 3 · 2	**2 (lista 3)**	1 → 1 → 2
4	4 · 3 · 6	**3 (lista 2)**	1 → 1 → 2 → 3

Como cada lista já está ordenada, o menor elemento de todos está sempre na cabeça de alguma lista. A cada passo, só as K cabeças precisam ser comparadas.

O DESAFIO

Casos de borda

Entrada	Resultado	O que isso exige
[]	vazio	Código não pode assumir que existe uma lista
[[]]	vazio	Uma lista pode ser só "nada"
[[], [1], []]	1	Listas vazias no meio devem ser ignoradas
[[1], [0,2,3,4,5]]	0 → 1 → 2 → 3 → 4 → 5	Uma lista acaba antes das outras

Quem esquece esses casos costuma errar justamente aqui.

ROTEIRO

Três formas de resolver

1	**Ordenar tudo**	A solução mais direta, mas que ignora a ordenação das listas
2	**Min-heap**	Sempre retirar o menor entre as K cabeças
3	**Divide and conquer**	Juntar as listas de duas em duas
4	**Comparação**	Tempo, espaço e quando usar cada uma

SOLUÇÃO 1 · ORDENAR TUDO

A ideia

A ideia mais direta: esquecer a estrutura de lista e tratar tudo como um simples array de números.

1. Percorrer todas as listas e juntar os valores num array
2. Ordenar o array
3. Montar uma lista ligada nova com os valores

O(N log N)
Tempo

O(N)
Espaço

Funciona e é correta, mas ignora que cada lista já vem ordenada. Estamos pagando para ordenar algo que já tinha ordem.

SOLUÇÃO 1 · ORDENAR TUDO

Coletando os valores

- Criamos um array vazio, vals, para guardar todos os valores.
- O for visita cada uma das K listas pela cabeça.
- O while anda até o fim da lista: guarda o valor e avança para o próximo nó.
- Cada nó é visitado uma vez, então esta etapa custa O(N).

```
def merge_k_lists(lists):
    vals = []
    for node in lists:
        while node:
            vals.append(node.val)
            node = node.next
```

SOLUÇÃO 1 · ORDENAR TUDO

Ordenando e reconstruindo

- vals.sort() ordena tudo. Este é o gargalo: O(N log N).
- dummy é um nó "falso" no início. Com ele, a cabeça não precisa de tratamento especial.
- Para cada valor ordenado criamos um nó novo, o que usa O(N) de memória.
- No fim devolvemos dummy.next, a cabeça verdadeira.

```
vals.sort()
dummy = cur = ListNode()
for v in vals:
    cur.next = ListNode(v)
    cur = cur.next
return dummy.next
```

SOLUÇÃO 2 · MIN-HEAP

A ideia

Se cada lista está ordenada, o menor elemento global está sempre na cabeça de alguma delas. Só precisamos olhar K candidatos.

1. Colocar a cabeça de cada lista no heap
2. Retirar o menor e anexar ao resultado
3. Colocar no heap o próximo nó daquela lista

O(N log K)
Tempo

O(K)
Espaço

O heap entrega o menor dos K candidatos em O(log K). Sem ele, teríamos que comparar as K cabeças a cada passo, O(K).

SOLUÇÃO 2 · MIN-HEAP

Preparando o heap

- O heap guarda tuplas (valor, índice, nó). Ele ordena pelo valor, então o menor fica no topo.
- O índice i desempata valores iguais, porque dois nós não podem ser comparados entre si.
- Só entram listas não vazias, e o heap nunca passa de K itens.
- dummy funciona como na solução anterior.

```
def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            push(heap, (node.val, i, node))
    dummy = cur = ListNode()
```

push = heapq.heappush · pop = heapq.heappop

SOLUÇÃO 2 · MIN-HEAP

O laço principal

- Enquanto houver candidatos, pop retira o menor: O(log K).
- Anexamos o próprio nó ao resultado, sem criar nenhum nó novo.
- Depois colocamos no heap o sucessor da mesma lista, se existir.
- Cada nó entra e sai do heap uma vez: N × O(log K).

```
    while heap:
        _, i, node = pop(heap)
        cur.next = node
        cur = node
        nxt = node.next
        if nxt:
            push(heap, (nxt.val, i, nxt))
    return dummy.next
```

SOLUÇÃO 3 · DIVIDE AND CONQUER

A ideia

Juntar duas listas ordenadas é simples e linear. Então basta reduzir K listas a pares, de forma repetida.

1. Juntar as listas de duas em duas
2. Repetir com as listas resultantes até sobrar uma

O(N log K)
Tempo

O(1)
Espaço extra

8 listas → 4 → 2 → 1. São log K rodadas e cada rodada percorre os N nós.

SOLUÇÃO 3 · DIVIDE AND CONQUER

Juntando duas listas

- Dois ponteiros, a e b, apontam para o começo de cada lista.
- O menor dos dois é ligado ao resultado, e esse ponteiro avança.
- Usar <= em vez de < preserva a ordem de valores iguais.
- Quando uma lista acaba, a or b anexa o resto da outra de uma vez.

```
def merge(a, b):
    dummy = cur = ListNode()
    while a and b:
        if a.val <= b.val:
            cur.next, a = a, a.next
        else:
            cur.next, b = b, b.next
        cur = cur.next
    cur.next = a or b
    return dummy.next
```

SOLUÇÃO 3 · DIVIDE AND CONQUER

Juntando as K listas

- step é a distância entre as listas que serão unidas.
- Rodada 1 (step = 1): une 0 com 1, 2 com 3, e assim por diante.
- Rodada 2 (step = 2): une 0 com 2, 4 com 6, e assim por diante.
- O step dobra a cada rodada, então são log K rodadas.
- O resultado fica em L[0]. Sem listas, devolvemos None.

```
def merge_k_lists(L):
    n, step = len(L), 1
    while step < n:
        for i in range(0, n - step, 2*step):
            L[i] = merge(L[i], L[i + step])
        step *= 2
    return L[0] if L else None
```

BALANÇO

Comparação

Abordagem	Tempo	Espaço	Ponto-chave
Ordenar tudo	O(N log N)	O(N)	Simples, ignora a ordenação
Min-heap	O(N log K)	O(K)	Fácil de explicar e implementar
Divide and conquer	O(N log K)	**O(1)**	Menos memória, reaproveita nós

Heap e divide and conquer têm o mesmo tempo. A diferença está no espaço extra.

O que levar daqui

1. Aproveite que as listas já estão ordenadas
2. Heap: sempre o menor das K cabeças
3. Divide and conquer: mesmo tempo, menos memória